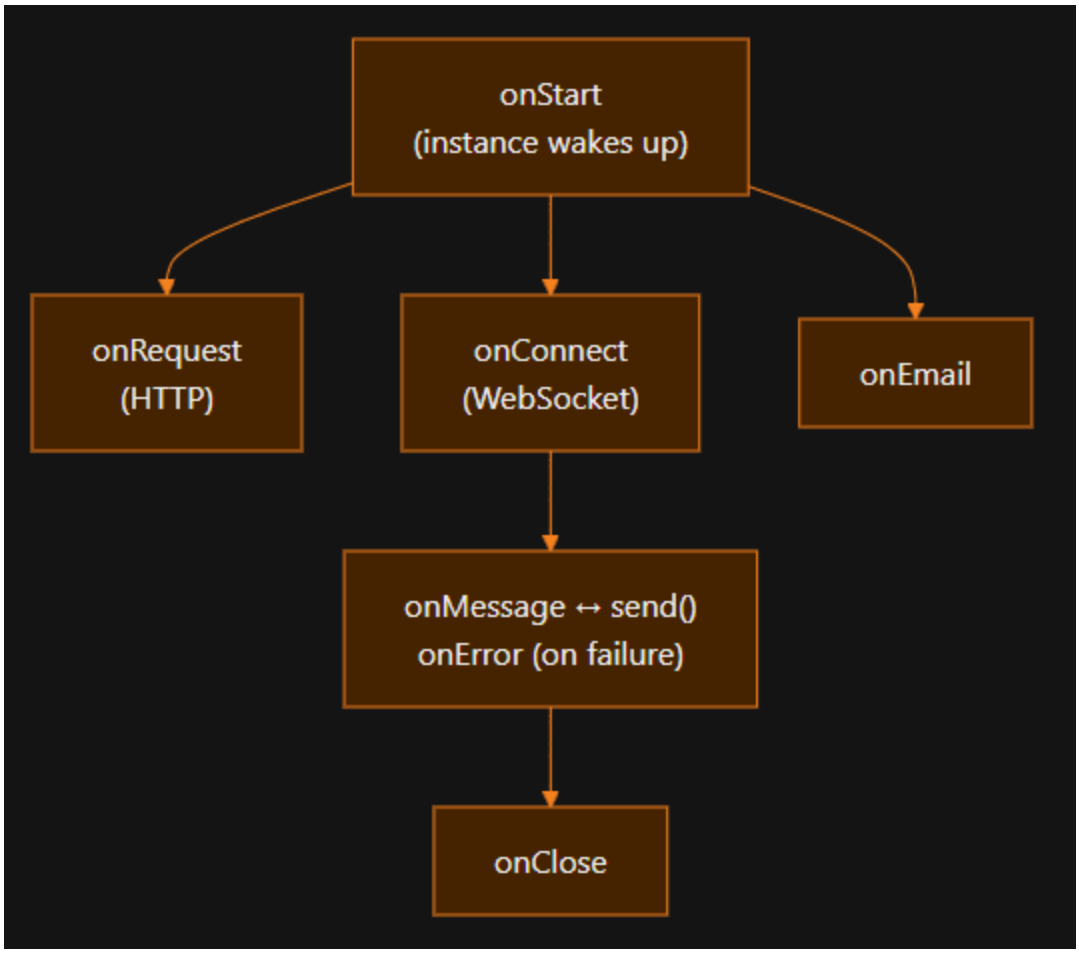
Milestone: 20GB Reached

We’ve reached 20GB of stored data — thank you for helping us grow!

Patreon Ko-fi Liberapay
Close
Freedium

- ko-fi
- librepay
- patreon

< Go to the original




# LangGraph vs Cloudflare Agents: Queues, Scheduling, and Durable Execution

## If you've built an LLM agent that does anything non-trivial, you've hit this moment:

Agent Native Follow
~12 min read · February 7, 2026 (Updated: February 7, 2026) · Free: No

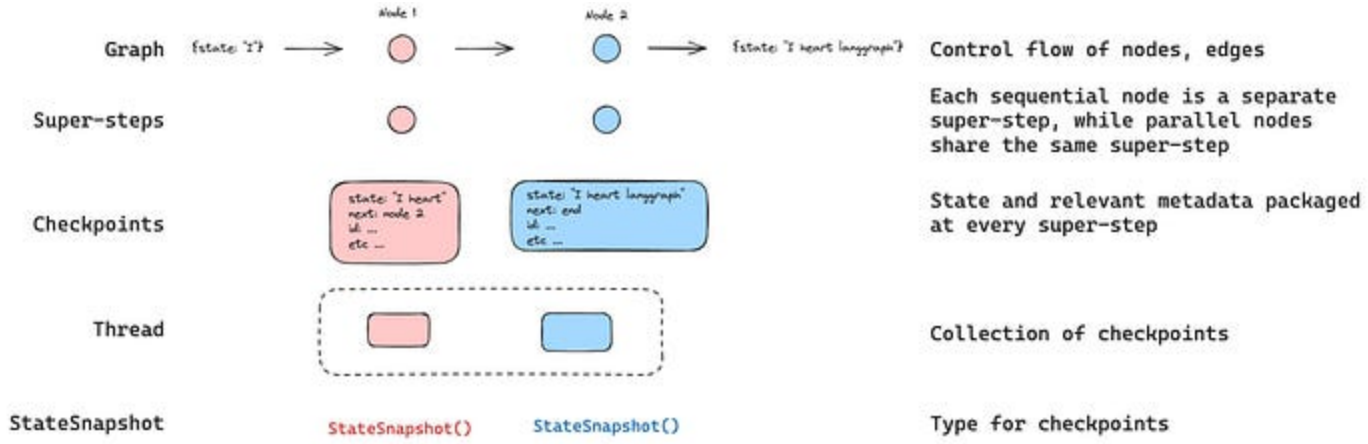
> *"Cool… but how do I make it do the thing **later**, **reliably**, and **without losing its place**?"*

That "later + reliably + resume" trio is basically:

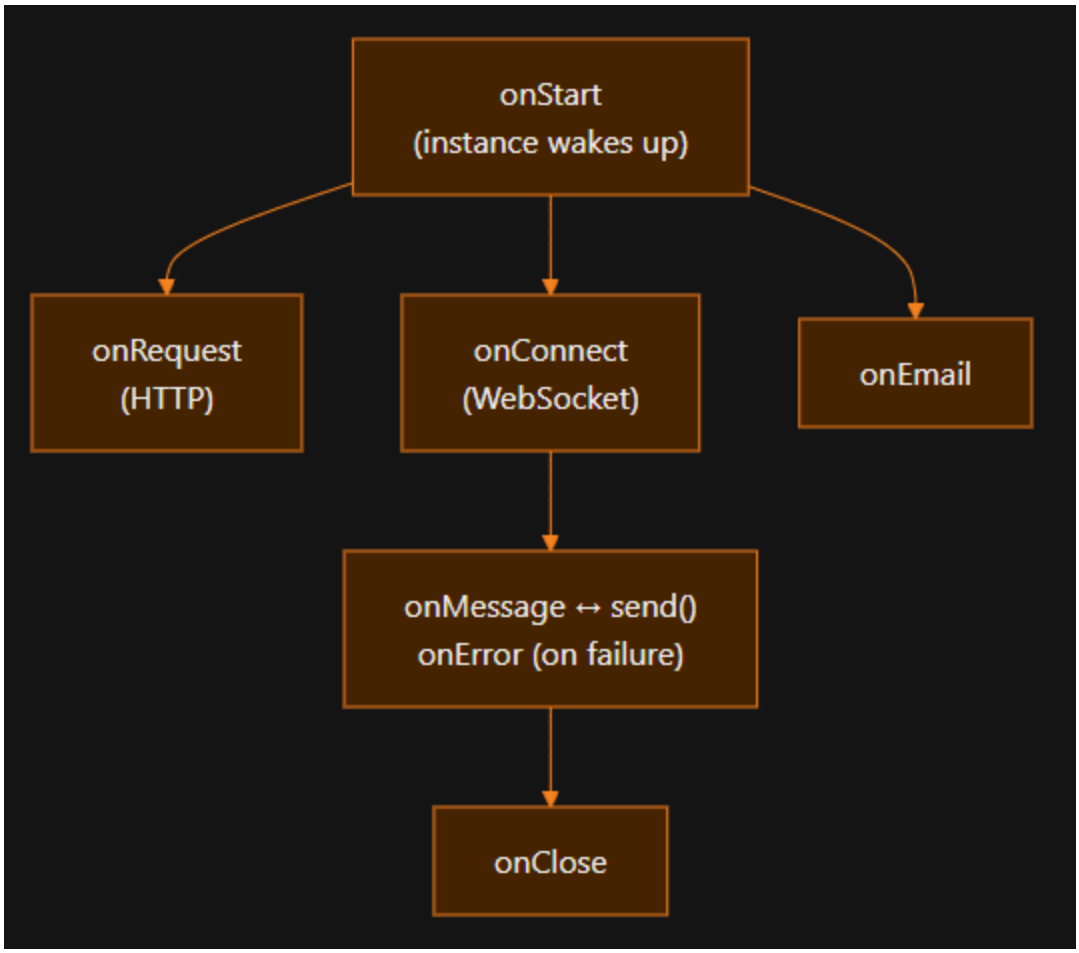
- **Queueing** (do work asynchronously)
- **Scheduling** (do work at a specific time / cron / interval)
- **Durable execution** (survive crashes, restarts, deploys, flaky APIs, timeouts)

**LangGraph** and **Cloudflare Agents** are both among your options, but they solve the problem from different layers of the stack.

- **LangGraph** is a workflow/orchestration framework for building stateful agent logic, with **checkpointing** and "resume from where you left off" baked into the execution model.




- **Cloudflare Agents** is an edge/serverless runtime + SDK for building stateful, real-time agents on top of **Durable Objects**, with built-in **task queueing** and **scheduling**, and optional **Workflows** for true durable, multi-step background runs.




So the right question usually isn't "which is better?" but:

> *"Do I need a **workflow engine**, a **serverless stateful runtime**, or both?"*

Let's break it down in a practical way, especially around queueing, scheduling, and durable execution.

> *If you're serious about building next-gen products with AI, I'm writing a deep-dive eBook on **Agentic SaaS**, the emerging design patterns that are quietly powering the most innovative startups of 2026.*
>
> ***Grab the first chapter free:** Agentic SaaS Patterns Winning in 2026, packed with real-world examples, architectures, and workflows you won't find anywhere else.*

## LangGraph: "my agent is a graph that checkpoints"

LangGraph runs your agent as a graph of nodes.

If you enable persistence (a **checkpointer**), it saves the graph state at each "super-step" to a **thread,** which unlocks resuming, time travel debugging, human-in-the-loop, etc.

```
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver
from langchain_core.runnables import RunnableConfig
from typing import Annotated
from typing_extensions import TypedDict
from operator import add

class State(TypedDict):
    foo: str
```

```
    bar: Annotated[list[str], add]

def node_a(state: State):
    return {"foo": "a", "bar": ["a"]}

def node_b(state: State):
    return {"foo": "b", "bar": ["b"]}


workflow = StateGraph(State)
workflow.add_node(node_a)
workflow.add_node(node_b)
workflow.add_edge(START, "node_a")
workflow.add_edge("node_a", "node_b")
workflow.add_edge("node_b", END)

checkpointer = InMemorySaver()
graph = workflow.compile(checkpointer=checkpointer)

config: RunnableConfig = {"configurable": {"thread_id": "1"}}
graph.invoke({"foo": "", "bar":[]}, config)
```

So durability happens at **node boundaries**.

If a node fails halfway through, a resume starts from the beginning of that node.

## Cloudflare Agents: "my agent is a stateful micro-server instance"

An Agent is a class running on Cloudflare **Durable Objects**.

Each agent instance is globally unique by ID, stateful, and you can have "millions of instances".

Durable Objects are **single-threaded** per instance (requests are processed sequentially, with async interleaving rules).

So you get a naturally stateful "agent-per-user (or per workspace/ticket/etc.)" model.

Copy

```
import { Agent, routeAgentRequest, callable } from "agents";

// Define the state shape
type CounterState = {
  count: number;
};

// Create the agent
export class Counter extends Agent<Env, CounterState> {
  // Initial state for new instances
  initialState: CounterState = { count: 0 };

  // Methods marked with @callable can be called from the client
  @callable()
  increment() {
    this.setState({ count: this.state.count + 1 });
    return this.state.count;
  }

  @callable()
  decrement() {
    this.setState({ count: this.state.count - 1 });
    return this.state.count;
  }

  @callable()
  reset() {
    this.setState({ count: 0 });
  }
}

// Route requests to agents
export default {
  async fetch(request: Request, env: Env, ctx: ExecutionContext) {
    return (
      (await routeAgentRequest(request, env)) ??
      new Response("Not found", { status: 404 })
    );
```

```
  },
};
```

And when you need durable background execution beyond "normal request time", you typically pair Agents with **Cloudflare Workflows**, which explicitly target durable, multi-step execution with retries and waiting for external events.

## Practical comparison

### (1) Queueing

#### Cloudflare Agents

- Built-in `queue()` that persists tasks to SQLite (`cf_agents_queues`) and processes them **sequentially (FIFO)**.
- No built-in retries/priority; you implement retry logic manually.
- Important limitation: "queue processing happens during agent execution, not as separate background jobs."

If you need a "real queue" decoupled from the agent instance, you use **Cloudflare Queues** (separate product).

It's reliable (messages aren't deleted until successfully consumed), but does **not guarantee publish order**.

Copy

```
class MyAgent extends Agent {
  async processEmail(data: { email: string; subject: string }) {
    // Process the email
    console.log(`Processing email: ${data.subject}`);
  }

  async onMessage(message: string) {
    // Queue an email processing task
    const taskId = await this.queue("processEmail", {
      email: "user@example.com",
      subject: "Welcome!",
    });

    console.log(`Queued task with ID: ${taskId}`);
  }
}
```

#### LangGraph

- LangGraph doesn't prescribe a queue and you can run graphs wherever (workers, Celery, Kubernetes jobs, etc.).
- If you deploy via "Agent Server / LangGraph API", you get **background runs** (async jobs with polling or webhook completion) and operational features built around that.

### (2) Scheduling

#### Cloudflare Agents

- `schedule()` supports delay / fixed date / cron, persists schedules to SQLite, survives restarts, and uses Durable Object **alarms** under the hood.

```
import { Agent } from "agents";

export class ReminderAgent extends Agent {
  async onRequest(request: Request) {
    const url = new URL(request.url);

    // Schedule in 30 seconds
    await this.schedule(30, "sendReminder", {
      message: "Check your email",
    });

    // Schedule at specific time
    await this.schedule(new Date("2025-02-01T09:00:00Z"), "sendReminder", {
      message: "Monthly report due",
    });

    // Schedule recurring (every day at 8am)
    await this.schedule("0 8 * * *", "dailyDigest", {
      userId: url.searchParams.get("userId"),
    });

    return new Response("Scheduled!");
  }

  async sendReminder(payload: { message: string }) {
    console.log(`Reminder: ${payload.message}`);
    // Send notification, email, etc.
  }

  async dailyDigest(payload: { userId: string }) {
    console.log(`Sending daily digest to ${payload.userId}`);
    // Generate and send digest
  }
}
```

- There's also `scheduleEvery()` for fixed-interval recurring tasks with overlap prevention (recent addition).

```
// Run every 5 minutes
await this.scheduleEvery("syncData", 5 * 60 * 1000, { source: "api" });
```

### LangGraph (Agent Server / LangGraph API)

- Supports **cron jobs** that run an assistant on a schedule (and you can control thread handling for stateless runs).
- The Python SDK exposes a client with a `crons` resource to create recurring runs.

```
# This schedules a job to run at 15:27 (3:27PM) UTC every day
cron_job = await client.crons.create_for_thread(
    thread["thread_id"],
    assistant_id,
    schedule="27 15 * * *",
    input={"messages": [{"role": "user", "content": "What time is it?"}]},
)
```

## (3) Durable execution

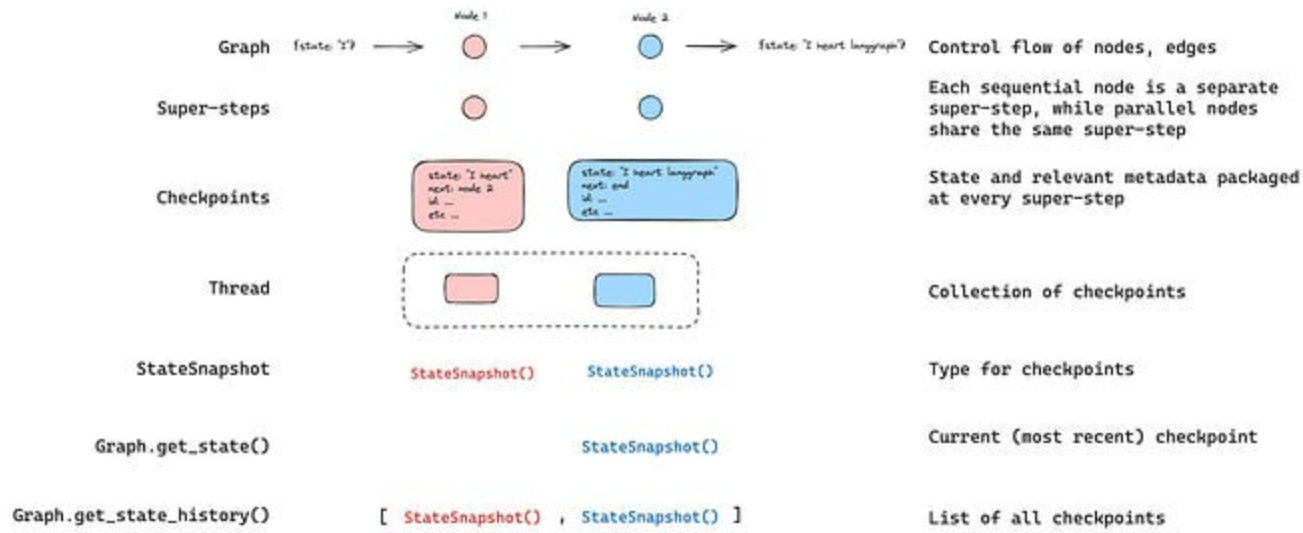
### LangGraph

- Checkpoints are saved at every super-step and state is stored to a thread, you can resume after interrupts/failures.
- Durability boundary is the node where resume restarts the node where execution stopped.

```
config = {"configurable": {"thread_id": "1"}}
list(graph.get_state_history(config))

# Output
[
    StateSnapshot(
```

```
        values={'foo': 'b', 'bar': ['a', 'b']},
        next=(),
        config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id': '1ef663ba-28fe-6528-8002-5
        metadata={'source': 'loop', 'writes': {'node_b': {'foo': 'b', 'bar': ['b']}}, 'step': 2},
        created_at='2024-08-29T19:19:38.821749+00:00',
        parent_config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id': '1ef663ba-28f9-6ec4
        tasks=(),
    ),
    StateSnapshot(
        values={'foo': 'a', 'bar': ['a']},
        next=('node_b',),
        config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id': '1ef663ba-28f9-6ec4-8001-3
        metadata={'source': 'loop', 'writes': {'node_a': {'foo': 'a', 'bar': ['a']}}, 'step': 1},
        created_at='2024-08-29T19:19:38.819946+00:00',
        parent_config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id': '1ef663ba-28f4-6b4a
        tasks=(PregelTask(id='6fb7314f-f114-5413-a1f3-d37dfe98ff44', name='node_b', error=None, interrupts=()),),
    ),
    StateSnapshot(
        values={'foo': '', 'bar': []},
        next=('node_a',),
        config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id': '1ef663ba-28f4-6b4a-8000-c
        metadata={'source': 'loop', 'writes': None, 'step': 0},
        created_at='2024-08-29T19:19:38.817813+00:00',
        parent_config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id': '1ef663ba-28f0-6c66
        tasks=(PregelTask(id='f1b14528-5ee5-579c-949b-23ef9bfbed58', name='node_a', error=None, interrupts=()),),
    ),
    StateSnapshot(
        values={'bar': []},
        next=('__start__',),
        config={'configurable': {'thread_id': '1', 'checkpoint_ns': '', 'checkpoint_id': '1ef663ba-28f0-6c66-bfff-6
        metadata={'source': 'input', 'writes': {'foo': ''}, 'step': -1},
        created_at='2024-08-29T19:19:38.816205+00:00',
        parent_config=None,
        tasks=(PregelTask(id='6d27aa2e-d72b-5504-a36f-8620e54a76dd', name='__start__', error=None, interrupts=()),)
    )
]
```




## Cloudflare Agents + Workflows

- Agents: durable state because Durable Objects persist storage and can be "woken up" via alarms.

Copy

```
import { DurableObject } from "cloudflare:workers";

export class AgentServer extends DurableObject {
  // Schedule a one-time or recurring event
  async scheduleEvent(id, runAt, repeatMs = null) {
    await this.ctx.storage.put(`event:${id}`, { id, runAt, repeatMs });
    const currentAlarm = await this.ctx.storage.getAlarm();
    if (!currentAlarm || runAt < currentAlarm) {
      await this.ctx.storage.setAlarm(runAt);
    }
  }

  async alarm() {
    const now = Date.now();
    const events = await this.ctx.storage.list({ prefix: "event:" });
    let nextAlarm = null;
```

```
    for (const [key, event] of events) {
      if (event.runAt <= now) {
        await this.processEvent(event);
        if (event.repeatMs) {
          event.runAt = now + event.repeatMs;
          await this.ctx.storage.put(key, event);
        } else {
          await this.ctx.storage.delete(key);
        }
      }
      // Track the next event time
      if (event.runAt > now && (!nextAlarm || event.runAt < nextAlarm)) {
        nextAlarm = event.runAt;
      }
    }

    if (nextAlarm) await this.ctx.storage.setAlarm(nextAlarm);
  }

  async processEvent(event) {
    // Your event handling logic here
  }
}
```

- Workflows: explicitly positioned as durable multi-step execution with retries, recovery, and waiting for external events; recommended for long-running tasks (e.g. >30s) and human approval flows.

Copy

```
import { AgentWorkflow } from "agents/workflows";
import type { AgentWorkflowEvent, AgentWorkflowStep } from "agents/workflows";
import type { MyAgent } from "./agent";

type TaskParams = { taskId: string; data: string };

export class ProcessingWorkflow extends AgentWorkflow<MyAgent, TaskParams> {
  async run(event: AgentWorkflowEvent<TaskParams>, step: AgentWorkflowStep) {
    const params = event.payload;

    const result = await step.do("process-data", async () => {
      return processData(params.data);
    });

    // Non-durable: progress reporting (may repeat on retry)
    await this.reportProgress({
      step: "process",
      status: "complete",
      percent: 0.5,
    });

    // Broadcast to connected WebSocket clients
    this.broadcastToClients({ type: "update", taskId: params.taskId });

    await step.do("save-results", async () => {
      // Call Agent methods via RPC
      await this.agent.saveResult(params.taskId, result);
    });

    // Durable: idempotent, won't repeat on retry
    await step.reportComplete(result);
    return result;
  }
}
```

The "durability unit" matters more than you think and this is the biggest gotcha people miss.

## LangGraph durability unit: node boundary

LangGraph checkpoints happen at node boundaries, and resuming starts at the beginning of the node where execution stopped.

So the design question becomes:

- Do you want **small nodes** (more checkpoints, less repeated work)?
- Or fewer big nodes (simpler graph, but more re-work on retry)?

LangGraph calls this out "node granularity trade-off" directly.

## Cloudflare Workflows durability unit: step

In Workflows, you typically wrap work in steps (`step.do(...)`), can sleep, retry, and wait for events.

That means you end up thinking like:

- "What's my **idempotent step**?"
- "What external side effects do I need to guard?"
- "Which steps are safe to retry automatically?"

## Example scenario: "agent that ingests docs, runs a pipeline, and asks for approval"

Let's use a realistic product-builder flow:

1. User uploads a doc (or drops a URL)
2. Agent parses + chunks + embeds + indexes
3. Agent generates a summary + action plan
4. Human approval gate before sending anything externally
5. Progress updates to the UI while it runs
6. Must survive deploys, restarts, and flaky APIs

## Cloudflare Agents + Workflows

Agents are great for realtime communication + per-user state, and Workflows are great for durable background pipelines and approval gates.

### Sketch architecture

- Browser connects to Agent via WebSocket
- Agent writes "job requested" to its durable state
- Agent starts a Workflow for ingestion + analysis
- Workflow emits progress, Agent broadcasts progress to clients
- Workflow pauses on approval using `waitForEvent`
- After approval, workflow continues and agent notifies UI

The docs explicitly describe "Agents excel at real-time… Workflows excel at durable execution… waiting for external events… human approval flows."

### Code draft:

Copy

```
import { Agent } from "agents";
import { MyIngestWorkflow } from "./workflows";

export class DocAgent extends Agent {
  async onMessage(connection, message) {
    const { type, docUrl } = JSON.parse(message);
    if (type === "ingest") {
      // Kick off durable workflow
      const workflow = await this.runWorkflow(MyIngestWorkflow, {
        docUrl,
        userId: this.state.userId,
      });
      connection.send(JSON.stringify({ type: "started", workflowId: workflow.id }));
```

```
    }
    if (type === "approve_send" && message.workflowId) {
      // Unblock workflow waiting for approval
      await this.sendWorkflowEvent(message.workflowId, "approval", { approved: true });
    }
  }
  async reportProgress(update) {
    // Broadcast progress to all connected clients
    this.broadcast(JSON.stringify({ type: "progress", ...update }));
  }
}
```

Inside the Workflow, you'll typically do:

- `step.do("parse", ...)`
- `step.do("embed", ...)`
- `step.sleep(...)` when needed
- `waitForEvent("approval")` for the human gate

This style is *fantastic* when your product is:

- real-time (streaming UX)
- user-scoped state (one agent per user/team)
- "runs in the background but I want live progress"

### Queueing & scheduling in this world

- For "do this soon but not now": `this.queue()` (FIFO, sequential implement retries yourself).
- For "do this later / cron / every N minutes": `this.schedule()` / `scheduleEvery()`, persisted to SQLite, backed by Durable Object alarms.

## LangGraph

LangGraph shines when your complexity is best expressed as a *graph* (routing, branching, tool loops, HITL checkpoints), and you want deep inspection/debugging via checkpoint history.

### The key primitives to understand (LangGraph durability)

- A **checkpointer** stores a checkpoint of state at every super-step to a **thread**.
- You invoke runs with `thread_id` so the engine can persist and resume.
- If execution stops mid-node, resume restarts that node.

### Minimal durable graph example:

Copy

```
from typing import TypedDict, List
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.postgres import PostgresSaver

class State(TypedDict):
    doc_url: str
    chunks: List[str]
    summary: str
    needs_approval: bool
    approved: bool

def parse_doc(state: State) -> State:
    # fetch + parse (external IO)
    chunks = ["chunk1", "chunk2"]
    return {"chunks": chunks}

def summarize(state: State) -> State:
    summary = f"Summary of {len(state['chunks'])} chunks"
```

```
    return {"summary": summary, "needs_approval": True}

def wait_for_approval(state: State) -> State:
    # In real systems, you interrupt and resume later with updated state.
    # (LangGraph supports interrupts/human-in-loop patterns via persistence.)
    if not state.get("approved"):
        raise RuntimeError("Not approved yet")
    return state

builder = StateGraph(State)
builder.add_node("parse", parse_doc)
builder.add_node("summarize", summarize)
builder.add_node("approval_gate", wait_for_approval)
builder.add_edge(START, "parse")
builder.add_edge("parse", "summarize")
builder.add_edge("summarize", "approval_gate")
builder.add_edge("approval_gate", END)

# Durable execution: persist checkpoints to Postgres
with PostgresSaver.from_conn_string("postgresql://...") as checkpointer:
    graph = builder.compile(checkpointer=checkpointer)

thread_id = "user-123/doc-456"
config = {"configurable": {"thread_id": thread_id}}
graph.invoke({"doc_url": "https://example.com/doc"}, config=config)
```

This example is intentionally simple, but it demonstrates the core durable mechanics:

- state checkpoints saved to a thread
- thread_id required for resume/time travel/hitl
- node boundary restart behavior

### Queueing & scheduling in LangGraph

If you run "just the library", you bring your own runtime primitives (Celery, job queues, cron, etc.).

But if you deploy via the LangGraph API / Agent Server, you get:

- **background runs** (async jobs with polling/webhooks)
- **cron jobs** (scheduled runs)

The Python SDK provides a `LangGraphClient` where you can create cron jobs like:

Copy

```
from langgraph_sdk import get_client

client = get_client(url="http://localhost:2024")

cron_job = await client.crons.create_for_thread(
    thread_id="thread_123",
    assistant_id="asst_456",
    schedule="0 9 * * *",
    input={"message": "Daily update"}
)
```

That "create_for_thread" example is shown directly in the SDK reference.

## Debuggability

LangGraph's checkpoint history is a superpower, once you embrace checkpointing, you can:

- inspect current and historical state
- replay from checkpoints
- do time-travel debugging
- implement "human-in-loop" cleanly

Also if you use the platform tooling, LangGraph Studio exists as a UI for visualizing/debugging runs locally.

## Cloudflare "unit of debugging" is the agent instance + workflow runs

Cloudflare Agents makes it very natural to debug:

- per-user agent instance state (because it's literally durable object state)
- workflow run progress/step boundaries (when you use Workflows)

But if you rely heavily on `queue()` inside the agent, remember it's FIFO/sequential, no built-in retry/priority, and runs during agent execution, not as an independent worker pool.

It's just a different operational model.

## Choosing between them

Choose Cloudflare Agents (plus Workflows) if you're building:

- a realtime product (chat, copilot UI, collaborative agent)
- per-user/per-workspace stateful experiences
- "push progress updates live while the job runs"
- an edge-first stack (Workers + Durable Objects)

You get scheduling out of the box (`schedule`, cron, intervals) with persistence and DO alarms.

And you get a clean story for "durable long jobs" via Workflows (retries, waiting, approval gates).

Choose LangGraph if your complexity is mainly:

- orchestration logic (branching, loops, tools, sub-agents)
- workflows you'll want to inspect/replay/debug over time
- Python-first ML/tooling stacks
- portability across clouds/runtimes

LangGraph's checkpointing model is central, super-step checkpoints stored to threads enable fault tolerance and more.

And the "node boundary" durability gives you a concrete design lever (node granularity).

## Pitfalls & best practices

### Idempotency isn't optional

- Cloudflare queue tasks explicitly recommend idempotent operations and note there's no built-in retry.
- LangGraph nodes may be re-run on resume (node-boundary restart).

So if a node/step sends an email, charges a card, writes to a 3rd-party system, guard it.

**Don't "accidentally serialize everything"**

- Cloudflare Durable Objects are single-threaded per instance. If you funnel too much work through one agent ID, you'll create a bottleneck.
- LangGraph can scale horizontally depending on where/how you run it, but you still need to think about concurrency per thread/run.

**Choose the right primitive for the job**

- **Cloudflare `queue()`**: "soon, sequential, inside agent execution."
- **Cloudflare Workflows**: "durable, long-running, multi-step, approval gates."
- **LangGraph checkpoints**: "workflow-level durability + introspection + replay."
- **LangGraph cron/background runs** (via API/server): "operational scheduling + async runs."

If your product's "magic" is:

- **the experience** (realtime, stateful UX, edge-native) → start with Cloudflare Agents (+ Workflows when it gets serious).
- **the orchestration logic** (graphs, routing, HITL, replay/debug) → start with LangGraph (and decide later whether you want to self-host or use the deployment layer).

### Bonus Articles

## Local LLMs That Can Replace Claude Code

**Small team of engineers can easily burn >$2K/mo on Anthropic's Claude Code (Sonnet/Opus 4.5). As budgets are tight, you...**

medium.com

## Founder's Open-Model Stack: GLM-4.7, Qwen3-VL, DeepSeek-V3.2, Kimi-K2, FLUX.2

**Founder's Open-Model Stack: GLM-4.7, Qwen3-VL, DeepSeek-V3.2, Kimi-K2, FLUX.2 If you're building an AI product as a...**

medium.com

## VIBESTACK: How Everyone Launches Products So Fast in 2026

**If you want to research, build, and launch your products fast, this is the solo founder stack for AI-native apps...**

medium.com

## "This PDF hates OCR" - DeepSeek OCR 2 Reads Documents Like A Human

### "This PDF hates OCR" - DeepSeek OCR 2 Reads Documents Like A Human Your customer forwards an invoice PDF with the...

medium.com

## Vectorless RAG for Agents: PageIndex Is Why Their Demo Works and Yours Needs “Context”

### When I’m building a RAG apps, the thing that quietly kills quality (and budgets) is turning a document into a pile of...

medium.com

## Fully Local Agentic Coding on localhost: Claude Code, Codex, llama.cpp, Ollama and Docker MR

### This is the setup I wish I had the first time I tried to run Claude Code and Codex-style workflows against local...

medium.com

## Open Source Perplexity: Perplexica is Self-Hosted AI Search With Citations

### Perplexica is an open-source, AI-powered search engine inspired by Perplexity-style UX.

medium.com

## Qwen3-Max-Thinking Outperforms Claude Opus 4.5 and Gpt-5.2: The Panic Is Real

### Alibaba’s Qwen3 family of models has become a major alternative in the LLM ecosystem.

medium.com
#cloudfare#langraph#durable-functions#ai-orchestration#agent-orchestration

# Reporting a Problem

Sometimes we have problems displaying some Medium posts.

If you have a problem that some images aren't loading - try using VPN. Probably you have problem with access to Medium CDN (or fucking Cloudflare's bot detection algorithms are blocking you).

Problem Description Describe your problem here...

Submit Cancel